

Chương này trình bày chi tiết năm đóng góp quan trọng nhất của đề án, bao gồm các giải pháp về nghiệp vụ, báo cáo thống kê và kỹ thuật. Mỗi đóng góp được trình bày theo cấu trúc: đặt vấn đề thực tiễn, phân tích giải pháp kỹ thuật đã triển khai và đánh giá kết quả đạt được trong hệ thống thực tế.

## **0.1 Đóng góp 1: Giải pháp xử lý đơn hàng đa kênh khép kín (Omnichannel)**

### **0.1.1 Giới thiệu và Đặt vấn đề**

Trong mô hình kinh doanh nhà hàng hiện đại, việc vận hành song song hai kênh bán hàng: tại chỗ (In-store) và trực tuyến (Online) thường gây ra sự phân mảnh dữ liệu nghiêm trọng. Nhân viên thu ngân phải sử dụng một phần mềm POS chuyên dụng để bán tại quầy, trong khi các đơn hàng trực tuyến lại đổ về một hệ thống quản lý riêng biệt hoặc thậm chí phải ghi nhận thủ công qua tin nhắn. Điều này dẫn đến ba vấn đề cốt lõi trong vận hành. Thứ nhất là sai lệch tồn kho (Inventory conflict), đặc biệt dễ xảy ra khi một món ăn có số lượng giới hạn được đặt mua cùng lúc từ cả hai kênh. Thứ hai là khó khăn trong công tác kế toán, khi người quản lý phải tổng hợp báo cáo tài chính từ nhiều nguồn dữ liệu rời rạc. Cuối cùng là áp lực lên bộ phận bếp, khi đầu bếp bị rối loạn bởi các luồng thông tin không đồng nhất, dẫn đến sai sót trong thứ tự chế biến món ăn.

### **0.1.2 Giải pháp kỹ thuật**

Để giải quyết triệt để các vấn đề trên, đề án đã thiết kế và triển khai một quy trình xử lý đơn hàng hợp nhất (Unified Order Processing) xoay quanh OrderService. Về mặt mô hình hóa dữ liệu, hệ thống sử dụng một thực thể Order duy nhất để đại diện cho tất cả các giao dịch, với sự phân biệt rõ ràng thông qua trường order\_type mang giá trị 'ONLINE' hoặc 'INSTORE'. Điều này tạo ra một cấu trúc dữ liệu nhất quán, đơn giản hóa việc truy vấn và quản lý.

Quan trọng hơn, hệ thống thiết lập một luồng xử lý tập trung cho mọi yêu cầu đặt hàng. Cả CreateOrderRequest từ ứng dụng khách hàng và CreateInstoreOrderRequest từ máy POS của thu ngân đều được điều hướng qua cùng một logic kiểm tra tồn kho nghiêm ngặt. Phương thức validateStock() trong ProductService đóng vai trò như một chốt chặn kiểm soát, sử dụng cơ chế khóa hàng (Row Lock) ở tầng cơ sở dữ liệu để đảm bảo tính toàn vẹn dữ liệu. Cơ chế này đảm bảo rằng tại bất kỳ thời điểm nào, một suất ăn cụ thể chỉ có thể được bán cho duy nhất một người dùng, loại bỏ hoàn toàn khả năng xung đột dữ liệu. Song song với đó, hệ thống tích hợp cơ chế đồng bộ thời gian thực với bộ phận bếp thông qua sự kiện WebSocket. Bất kể đơn hàng xuất phát từ nguồn nào, thông tin món ăn đều được đẩy tức thì tới màn hình hiển thị của nhà bếp, giúp đầu bếp dễ dàng sắp xếp công việc theo nguyên tắc FIFO (Nhập trước Xuất trước).

### **0.1.3 Kết quả đạt được**

Việc triển khai giải pháp này đã tạo ra một hệ thống quản lý vận hành liền mạch và hiệu quả. Nhân viên thu ngân giờ đây có thể giám sát toàn bộ hoạt động kinh doanh, bao gồm cả các đơn hàng online, ngay trên giao diện POS quen thuộc mà không cần chuyển đổi qua lại giữa các ứng dụng. Tồn kho nguyên liệu được trừ tự động và chính xác theo thời gian thực, triệt tiêu hoàn toàn tình trạng bán vượt quá số lượng thực tế. Đặc biệt, quy trình báo cáo doanh thu cuối ngày được tự động hóa hoàn toàn, tích hợp dòng tiền từ cả hai kênh vào một báo cáo duy nhất. Cải tiến này giúp người quản lý tiết kiệm được từ 30 đến 60 phút mỗi ngày cho công tác đối soát, đồng thời giảm thiểu tối đa các sai sót do tính toán thủ công.

## **0.2 Đóng góp 2: Báo cáo thống kê doanh thu ca làm việc chính xác**

### **0.2.1 Giới thiệu và Đặt vấn đề**

Tại các cửa hàng cơm bình dân và dịch vụ ăn uống nhỏ lẻ, việc quản lý dòng tiền mặt luôn là một thách thức lớn, thường được ví như điểm yếu chí tử trong quản lý tài chính. Do đặc thù nhân viên thu ngân làm việc theo ca (Shift), quá trình bàn giao tiền mặt giữa các ca thường xuyên xảy ra sai sót, nhầm lẫn hoặc thậm chí là gian lận. Chủ cửa hàng đối mặt với khó khăn trong việc xác minh xem số tiền thực tế trong két có khớp đúng với doanh thu bán hàng được ghi nhận hay không. Vấn đề càng trở nên phức tạp hơn trong bối cảnh thanh toán không dùng tiền mặt (như chuyển khoản VietQR) ngày càng phổ biến, dẫn đến việc lẫn lộn giữa luồng tiền mặt và luồng tiền ngân hàng.

### **0.2.2 Giải pháp kỹ thuật**

Hệ thống giải quyết bài toán này bằng việc xây dựng module quản lý phiên làm việc (Shift Management) gắn liền chặt chẽ với mọi giao dịch tài chính phát sinh. Trung tâm của giải pháp là thiết kế thực thể Shift, lưu trữ đầy đủ các thông tin kiểm soát bao gồm thời gian bắt đầu, thời gian kết thúc, nhân viên phụ trách và số dư đầu ca.

Điểm đột phá nằm ở khả năng tự động phân loại dòng tiền thông minh. Mỗi khi một đơn hàng được thanh toán thành công, hệ thống sẽ căn cứ vào trường `payment_method` để phân bổ doanh thu vào đúng hạng mục của ca làm việc hiện tại. Cụ thể, doanh thu sẽ được cộng dồn vào `cash_revenue` nếu khách trả tiền mặt, hoặc vào `transfer_revenue` nếu khách chuyển khoản. Logic nghiệp vụ này được thực thi trong phương thức `OrderService.processPayment()`, đảm bảo tính nguyên tử của giao dịch, tức là tiền chỉ được ghi nhận khi đơn hàng chắc chắn đã thành công. Ngoài ra, quy trình chốt ca cũng được tự động hóa. Khi nhân viên thực hiện thao tác "Kết thúc ca", hệ thống sẽ lập tức khóa phiên làm việc hiện tại, tính toán

tổng doanh thu theo từng loại hình thanh toán và so sánh đối chiếu với số tiền nhân viên khai báo thực tế. Bất kỳ khoản chênh lệch nào cũng sẽ được hệ thống phát hiện và cảnh báo ngay lập tức.

### **0.2.3 Kết quả đạt được**

Kết quả của giải pháp là sự ra đời của tính năng báo cáo chốt ca (Shift Report) minh bạch và tức thời. Ngay khi kết thúc phiên làm việc, chủ cửa hàng có thể nắm bắt chính xác tình hình tài chính với độ chi tiết cao, ví dụ: "Doanh thu ca sáng là 5 triệu đồng, trong đó 3 triệu đồng là tiền mặt cần thu về kết, và 2 triệu đồng đã được chuyển vào tài khoản ngân hàng". Sự rõ ràng này giúp loại bỏ hoàn toàn sự nhập nhằng trong quản lý tài chính, tăng cường tính trung thực và trách nhiệm giải trình của nhân viên, đồng thời đơn giản hóa đáng kể công tác kế toán cho chủ doanh nghiệp.

## **0.3 Đóng góp 3: Phân tích xu hướng ẩm thực và tối ưu thực đơn**

### **0.3.1 Giới thiệu và Đặt vấn đề**

Lãng phí thực phẩm là một trong những nguyên nhân hàng đầu gây thất thoát lợi nhuận và tăng chi phí vận hành trong kinh doanh ăn uống. Việc nấu quá nhiều các món ăn ít người gọi dẫn đến tình trạng ế thừa phải đổ bỏ, trong khi việc chuẩn bị quá ít các món "best-seller" lại làm mất cơ hội bán hàng và giảm sự hài lòng của thực khách. Tuy nhiên, thực tế cho thấy hầu hết các chủ cửa hàng quy mô nhỏ vẫn đang ra quyết định nhập nguyên liệu và lên thực đơn dựa trên cảm tính hoặc thói quen, thay vì dựa trên các dữ liệu bán hàng thực tế.

### **0.3.2 Giải pháp kỹ thuật**

Để khắc phục tình trạng này, đồ án cung cấp một bộ công cụ phân tích dữ liệu bán hàng thông minh (Analytical Dashboard) dựa trên lịch sử giao dịch. Về mặt kỹ thuật, hệ thống sử dụng các truy vấn JPQL/SQL phức tạp kết hợp với các hàm tổng hợp (Aggregate Functions) để khai thác dữ liệu từ bảng chi tiết đơn hàng `order_items`. Hệ thống thực hiện tính toán tần suất xuất hiện của từng mã sản phẩm (`product_id`) trong các khoảng thời gian tùy chọn như 7 ngày hoặc 30 ngày gần nhất.

Để đảm bảo hiệu năng truy vấn cho các báo cáo được truy cập thường xuyên, đồ án sử dụng kỹ thuật tạo View cơ sở dữ liệu với tên gọi `top_selling_products`. View này đóng vai trò như một bảng ảo được tối ưu hóa, giúp giảm tải việc tính toán lại mỗi khi có yêu cầu báo cáo. Dữ liệu sau khi xử lý được API trả về dưới định dạng chuẩn JSON và được Frontend trực quan hóa sinh động. Người dùng được cung cấp các biểu đồ cột và biểu đồ tròn trực quan, làm nổi bật rõ ràng danh sách Top 5 món ăn bán chạy nhất cũng như Top 5 món có doanh thu thấp nhất trong kỳ.

### 0.3.3 Kết quả đạt được

Công cụ phân tích này đóng vai trò như một trợ lý ảo đặc lực, hỗ trợ chủ của hàng trong việc tư vấn và tối ưu hóa thực đơn. Thay vì phỏng đoán, giờ đây người quản lý có thể nhìn vào các biểu đồ trực quan và đưa ra các quyết định chính xác dựa trên số liệu. Ví dụ, họ có thể dễ dàng nhận thấy món "Sườn xào chua ngọt" luôn hết sớm vào các ngày thứ Hai để chủ động tăng lượng nhập, hoặc nhận ra món "Cá kho tộ" có sức tiêu thụ kém trong tuần qua để điều chỉnh giảm số lượng chế biến. Kết quả thực nghiệm cho thấy việc áp dụng giải pháp này đã giúp giảm lượng thức ăn thừa cuối ngày khoảng 15% đến 20% chỉ sau một tháng vận hành.

## 0.4 Đóng góp 4: Cơ chế bảo mật đa lớp với JWT (Jason Web Token)

### 0.4.1 Giới thiệu và Đặt vấn đề

Trong phát triển ứng dụng web hiện đại, việc đảm bảo an toàn thông tin và kiểm soát truy cập là ưu tiên hàng đầu. Các ứng dụng truyền thống thường dựa vào cơ chế Session lưu trữ trên máy chủ để quản lý trạng thái đăng nhập. Tuy nhiên, với kiến trúc tách biệt hoàn toàn giữa Frontend (ReactJS) và Backend (Spring Boot), việc sử dụng Session trở nên cồng kềnh, khó khăn khi mở rộng hệ thống (Scaling) và không tương thích tốt với các nền tảng di động. Bên cạnh đó, các giao diện lập trình ứng dụng (API) cần được bảo vệ nghiêm ngặt để ngăn chặn các truy cập trái phép vào dữ liệu nhạy cảm hoặc các chức năng quản trị quan trọng.

### 0.4.2 Giải pháp kỹ thuật

Hệ thống đã triển khai một cơ chế xác thực phi trạng thái (Stateless Authentication) mạnh mẽ dựa trên chuẩn công nghiệp JWT (JSON Web Token). Cốt lõi của cơ chế này là bộ lọc bảo mật JwtAuthenticationFilter được tích hợp sâu vào quy trình xử lý của Spring Security. Bộ lọc này có nhiệm vụ chặn mọi yêu cầu (Request) gửi đến API, thực hiện giải mã chuỗi Token trong Header, xác thực chữ ký điện tử (Signature) để đảm bảo tính toàn vẹn và trích xuất thông tin định danh người dùng.

Song song với xác thực, hệ thống áp dụng mô hình phân quyền dựa trên vai trò (Role-Based Access Control - RBAC) để kiểm soát quyền hạn chi tiết. Spring Security được cấu hình linh hoạt để bảo vệ ở mức phương thức (Method Level Security). Ví dụ, annotation `@PreAuthorize("hasRole('ADMIN')")` được áp dụng triệt để cho các API quản lý nhân sự, đảm bảo tuyệt đối rằng nhân viên thu ngân dù có Token hợp lệ cũng không thể thực hiện các hành động vượt quyền như xóa tài khoản người dùng khác. Về bảo mật dữ liệu, mật khẩu người dùng không bao giờ được lưu trữ dưới dạng văn bản thô (Plain text). Thay vào đó, hệ thống sử dụng thuật toán băm mật khẩu một chiều BCrypt kết hợp với cơ chế Salt ngẫu nhiên, tạo

ra một lớp bảo vệ vững chắc ngay cả trong trường hợp cơ sở dữ liệu bị lộ lọt.

### **0.4.3 Kết quả đạt được**

Việc áp dụng các giải pháp trên đã mang lại cho hệ thống một cấp độ bảo mật cao và linh hoạt. Việc sử dụng JWT cho phép Frontend dễ dàng lưu trữ và quản lý trạng thái đăng nhập ngay trên trình duyệt (thông qua LocalStorage) mà không phụ thuộc vào tài nguyên máy chủ, giúp hệ thống dễ dàng mở rộng. Các API được bảo vệ an toàn trước các nguy cơ tấn công giả mạo hoặc truy cập trái phép. Quan trọng nhất, cơ chế này đảm bảo rằng dữ liệu nhạy cảm và các chức năng quan trọng của hệ thống chỉ được tiếp cận bởi đúng người, đúng thẩm quyền, tạo dựng niềm tin tuyệt đối cho người sử dụng.

## **0.5 Đóng góp 5: Kiến trúc phân tầng (Layered Architecture) và DTO Pattern**

### **0.5.1 Giới thiệu và Đặt vấn đề**

Trong quy trình phát triển phần mềm, một trong những sai lầm phổ biến nhưng để lại hậu quả nghiêm trọng là việc tổ chức mã nguồn thiếu khoa học, hay còn gọi là "Spaghetti code". Việc trộn lẫn logic xử lý giao diện, logic nghiệp vụ phức tạp và các câu lệnh truy vấn cơ sở dữ liệu vào chung một nơi khiến hệ thống trở nên rối rắm, khó bảo trì và cực kỳ khó mở rộng. Ngoài ra, một rủi ro bảo mật thường gặp khác là việc trả về trực tiếp các thực thể cơ sở dữ liệu (Entity) ra ngoài giao diện người dùng. Điều này không chỉ vô tình làm lộ các thông tin nội bộ nhạy cảm như cấu trúc bảng, mật khẩu, ngày tạo... mà còn dễ gây ra lỗi vòng lặp vô tận (Infinite Recursion) trong quá trình chuyển đổi dữ liệu sang định dạng JSON.

### **0.5.2 Giải pháp kỹ thuật**

Để khắc phục triệt để các vấn đề trên, đồ án đã tuân thủ nghiêm ngặt mô hình kiến trúc phân tầng (Layered Architecture) chuẩn mực với ba tầng riêng biệt: Controller, Service và Repository. Trong đó, tầng Controller chỉ chịu trách nhiệm tiếp nhận yêu cầu và điều hướng phản hồi. Toàn bộ logic nghiệp vụ, tính toán và các quy tắc kiểm tra ràng buộc được cô lập hoàn toàn trong tầng Service. Tầng Repository đảm nhận vai trò duy nhất là giao tiếp và truy xuất dữ liệu từ Database. Sự phân tách rạch ròi này giúp mã nguồn trở nên trong sáng và dễ dàng kiểm thử.

Bên cạnh kiến trúc phân tầng, hệ thống áp dụng triệt để mẫu thiết kế Đối tượng chuyển dữ liệu (Data Transfer Object - DTO). Đồ án định nghĩa hệ thống các lớp DTO riêng biệt cho cả chiều dữ liệu vào (Request DTO) và chiều dữ liệu ra (Response DTO), ví dụ như OrderResponse hay UserDTO. Dữ liệu từ các Entity sẽ được "chiết xuất" và chuyển đổi sang các DTO tương ứng trước khi được trả về cho Client. Quá trình này giúp loại bỏ hoàn toàn các trường dữ liệu nhạy cảm hoặc không cần thiết như password hay created\_by khỏi kết quả trả về. Để đảm bảo sự

nhất quán và giảm thiểu sai sót, các phương thức chuyển đổi (Mapper) được sử dụng để ánh xạ dữ liệu giữa Entity và DTO một cách tự động và chuẩn xác.

### **0.5.3 Kết quả đạt được**

Kết quả thu được là một bộ mã nguồn (Source code) có chất lượng cao, "sạch", dễ đọc hiểu và thuận tiện cho việc bảo trì, nâng cấp. Khi có yêu cầu thay đổi logic nghiệp vụ, ví dụ như thay đổi cách tính giá khuyến mãi, lập trình viên chỉ cần tập trung sửa đổi tại tầng Service mà không làm ảnh hưởng tới Controller hay cấu trúc cơ sở dữ liệu. Đặc biệt, việc áp dụng DTO giúp hệ thống kiểm soát hoàn toàn định dạng và nội dung dữ liệu trả về cho Client, qua đó nâng cao đáng kể tính bảo mật và tối ưu hóa băng thông mạng nhờ việc chỉ truyền tải những dữ liệu thực sự cần thiết.